

LogSfBM_Model

Log-Stationary Fractional Brownian Motion

Simulation and statistical analysis — Technical Documentation

Othmane Zarhali

June 17, 2026

Abstract

This document provides a complete mathematical and software reference for the **LogSfBM_Model** package, which implements the Log-Stationary fractional Brownian Motion (Log-SfBM) model of rough stochastic volatility. It covers two modules in full: **LogSfBM_Class.py**, the core simulation class implementing the univariate and multivariate Log-SfBM driving process, the Wood–Chan FFT Gaussian process simulator, and the associated multifractal price/variance paths; and **LogSfBM_StatisticalProperties.py**, a companion module implementing moment-scaling analysis, autocovariance diagnostics, OHLC log-volatility estimators, and simulated-versus-theoretical comparison tools. Every function and method is documented with its exact mathematical definition, the corresponding implementation, and any known numerical caveats discovered during testing.

Contents

1	Mathematical Background	3
1.1	Driving process covariance	3
1.2	Price and variance processes	3
1.3	Multivariate extension	3
2	Module: LogSfBM_Class.py — Core Simulation Class	5
2.1	GaussianProcessSimulation(covariance, size)	5
2.2	Class LogSfBM	5
2.2.1	Constructor: __init__	5
2.2.2	CovarianceFunction_SfBM	6
2.2.3	CrossAutocovariance_mSfBM	6
2.2.4	_sfbmomcorr	6
2.2.5	GenerateSfBM_btwttimebounds	6
2.2.6	Generate_IntegratedSfBM	6
2.2.7	LogSfBM_Simulation	6
2.2.8	LogSfBM_Nested_Simulation	7
2.2.9	genlogVol & genlogVol_New_perscale	8
3	Module: LogSfBM_StatisticalProperties.py — Statistical Diagnostics	9
3.1	Increment generation	9
3.1.1	Increments	9
3.2	Distributional diagnostics	9
3.2.1	LogIncrementsDistributionDensityAcrossScale	9
3.3	Multifractal moment-scaling diagnostics	9
3.3.1	MomentIncrementsRepresentation	9
3.3.2	MomentIncrementsRepresentationSimulatedVSTheoretical	10

3.3.3	MomentIncrementsEvolutionWrtParameter	10
-------	---	----

1 Mathematical Background

The Log-SfBM model describes a continuous-time stochastic volatility process driven by a centred, stationary Gaussian log-volatility field ω_t . The model is parametrised by three quantities:

- $H \in (0, \frac{1}{2})$ — the **Hurst exponent**, controlling the roughness of the volatility path (smaller H produces rougher paths);
- $\lambda^2 > 0$ — the **intermittency coefficient**, controlling the variance of the log-volatility field;
- $T > 0$ — the **integral (decorrelation) scale**, beyond which the covariance of ω_t vanishes.

1.1 Driving process covariance

The log-volatility field ω_t is a centred stationary Gaussian process with covariance

$$C_\omega(t, s) = \frac{\lambda^2}{2H(1-2H)} \left[1 - \frac{|t-s|^{2H}}{T^{2H}} \right] \mathbf{1}_{|t-s| < T}. \quad (1)$$

This is a function of the lag $\tau = |t-s|$ only (stationarity), has compact support $|\tau| < T$, and the constant prefactor

$$K = \frac{\lambda^2}{2H(1-2H)}$$

gives the process variance $C_\omega(0) = K$.

1.2 Price and variance processes

From ω_t , two associated processes are constructed:

$$X_t = \int_0^t e^{\omega_s/2} dW_s \quad (\text{MRW — Multifractal Random Walk, the price path}), \quad (2)$$

$$M_t = \sigma^2 \int_0^t e^{\omega_s} ds \quad (\text{MRM — Multifractal Random Measure, the integrated variance}). \quad (3)$$

A mean correction $m = -\lambda^2/[4H(1-2H)]$ is added to every simulated ω_t path so that $\mathbb{E}[e^{\omega_t}] = 1$, ensuring M_t is a properly normalised measure.

1.3 Multivariate extension

The d -dimensional generalisation replaces the scalar pair (H, λ^2) with two symmetric $d \times d$ matrices:

- the **co-Hurst matrix** H_{ij} , giving the cross-scaling exponent between components i and j (diagonal entries H_{ii} recover the marginal Hurst exponents);

- the **cointermittency matrix** Λ_{ij} , giving the cross-intermittency between components i and j .

For the bivariate case ($d = 2$), the cross-covariance between component 1 and component 2 at lag τ is

$$C_{12}(\tau) = \lambda_{12} \xi_{12} \left[\frac{1}{2H_{12}} - \frac{1}{2\bar{H} - 1} + \frac{|\tau|}{T} \left(\frac{-1}{2H_{12} - 1} + \frac{1}{2\bar{H} - 1} \right) + \left(\frac{|\tau|}{T} \right)^{2H_{12}} \left(\frac{1}{2H_{12} - 1} - \frac{1}{2H_{12}} \right) \right] \mathbf{1}_{|\tau| \leq T}, \quad (4)$$

where $\bar{H} = (H_{11} + H_{22})/2$ is the mean marginal Hurst exponent, H_{12} the off-diagonal co-Hurst exponent, $\lambda_{12} = \sqrt{\Lambda_{11}\Lambda_{22}}$ the geometric mean of the marginal intermittency coefficients, and $\xi_{12} = \Lambda_{12}$ the off-diagonal cointermittency coefficient.

2 Module: LogSfBM_Class.py — Core Simulation Class

This module implements the `LogSfBM_Class` class together with one module-level helper, `GaussianProcessSimulation`, that performs exact stationary Gaussian process simulation via the Wood–Chan circulant-embedding algorithm.

2.1 `GaussianProcessSimulation(covariance, size)`

Simulates a stationary Gaussian process with a prescribed autocovariance sequence $c_0, c_1, \dots, c_{K-1}$ using the Wood–Chan (circulant embedding) method, which is *exact* (matches the prescribed covariance to machine precision) whenever the embedding circulant has non-negative eigenvalues.

Algorithm.

1. Choose $m = 2^{\lceil \log_2(\text{size}) \rceil}$ and set the circulant block length $M = 2m$.
2. Zero-pad c to length $m + 1$ and build the circulant row

$$r = [c_0, c_1, \dots, c_m, c_{m-1}, \dots, c_1] \quad (\text{length } M).$$

3. Compute the eigenvalues of the circulant via the discrete Fourier transform: $\Lambda_k = \text{FFT}(r)_k$.
4. Draw i.i.d. standard normals u_k, v_k and form the complex frequency-domain vector

$$Z_k = \sqrt{\Lambda_k} \frac{u_k + iv_k}{\sqrt{2}}.$$

5. Return the real path

$$x = \text{Re}(\text{IFFT}(Z)) \cdot \sqrt{2M},$$

truncated to the first `size` entries.

Note

Small negative eigenvalues caused by floating-point rounding are absorbed harmlessly by the complex square root $\sqrt{\Lambda_k + 0i}$: the resulting imaginary contribution is discarded by the final $\text{Re}(\cdot)$.

2.2 Class `LogSfBM`

2.2.1 Constructor: `__init__(H, lambda2, T, coHurst_matrix, cointermittency_matrix)`

Two mutually exclusive construction paths:

- **Univariate** (`coHurst_matrix` and `cointermittency_matrix` both `None`): `H` and `lambda2` must be scalars; sets `dimension = 1`.
- **Multivariate** (at least one matrix supplied): both matrices, if given, must be square, symmetric, and of matching dimension; `H` and `lambda2` are then unused (set to `None`) and `dimension` is read from the matrix shape.

Validation failures raise `TypeError` (wrong scalar/matrix usage) or `ValueError` (non-square, non-symmetric, or mismatched-dimension matrices).

2.2.2 CovarianceFunction_SfBM(t, s, marginal=None)

Implements Equation (1) directly:

$$C_\omega(t, s) = \frac{\lambda^2}{2H(1-2H)} \left[1 - \frac{|t-s|^{2H}}{T^{2H}} \right] \mathbf{1}_{|t-s|<T}.$$

When `marginal=k` is supplied (multivariate case), (H, λ^2) is replaced by the diagonal entries (H_{kk}, Λ_{kk}) , recovering the marginal covariance of component k .

2.2.3 CrossAutocovariance_mSfBM(tau, marginal=None)

Implements Equation (4) for the bivariate case ($d = 2$ required, else raises `ValueError`). When `marginal=k` is given instead of `None`, the function returns the diagonal (marginal) covariance $C_{kk}(\tau)$ using the same parameter-extraction logic as the cross-covariance case.

2.2.4 _sfbmomcorr(size, dt, T, lambda2, H)

Builds the discretised autocovariance sequence of ω_t at step dt , for $k = 0, \dots, \text{size} - 1$:

$$H \neq 0 : \quad c_k = \frac{\lambda^2}{2H(1-2H)} \left[1 - \frac{(k \, dt)^{2H}}{(T \, dt)^{2H}} \right] \mathbf{1}_{k \, dt < T \, dt}, \quad (5)$$

$$H = 0 \text{ (MRW case)} : \quad c_k = \lambda^2 \log\left(\frac{T}{k \, dt}\right) \mathbf{1}_{k \, dt < T}, \quad c_0 = \lambda^2 \left(1 + \log \frac{T}{dt}\right). \quad (6)$$

The function also returns the mean-correction term $m = -c_0$, which shifts the simulated path so that $\mathbb{E}[e^{\omega_t}] = 1$.

2.2.5 GenerateSfBM_btwttimebounds(t_min, t_max, size, Msubsample)

Simulates ω_t on `size` equally-spaced points of $[t_{\min}, t_{\max}]$, using the covariance vector $C_\omega(t_{\min}, t_k)$ for each grid point t_k (anchored at the left boundary), then applies the mean correction

$$m = -\frac{\lambda^2 T^{2H}}{4H(1-2H)}.$$

2.2.6 Generate_IntegratedSfBM(size, dt)

Simulates the integrated log-volatility process Ω_t , whose autocovariance is

$$C_\Omega(\tau) = \gamma_1 - \gamma_2 \left[|\tau + 1|^{2H+2} + |\tau - 1|^{2H+2} - 2|\tau|^{2H+2} \right], \quad (7)$$

with

$$\gamma_1 = \frac{T^{2H}}{2H(1-2H)}, \quad \gamma_2 = \frac{1}{2H(1-(2H)^2)(2+2H)}.$$

2.2.7 LogSfBM_Simulation(size, subsample, sigma, flagm, returnomega, returnOmega, simulation_method)

The principal simulation routine, with separate univariate and multivariate code paths.

Univariate case. With $dt = 1/\text{subsample}$:

1. Build the fine-grid covariance sequence of ω via `_sfbmomcorr` (Eqs. (5)–(6)).
2. Simulate $\omega_t \sim \text{GP}(0, C_\omega)$ via `GaussianProcessSimulation`, then shift by $m = -\lambda^2/[4H(1-2H)]$.
3. Construct, with i.i.d. $g_k \sim \mathcal{N}(0, 1)$ and $dt = 1/\text{subsample}$:

$$X_t = \sum_{k=0}^{t/dt-1} e^{\omega_k/2} g_k \sigma \sqrt{dt} \quad (\text{MRW}), \quad (8)$$

$$M_t = \sigma^2 \sum_{k=0}^{t/dt-1} e^{\omega_k} dt \quad (\text{MRM}). \quad (9)$$

Both are sub-sampled back to the coarse grid (every `subsample`-th point). Optional flags select alternate return values: `returnomega=True` returns the raw fine-grid ω ; `returnOmega=True` returns the block-integrated, mean-centred process

$$\Omega_j = \frac{1}{\sqrt{\lambda^2}} \sum_{k \in \text{block}_j} \omega_k dt, \quad \Omega \mapsto \Omega - \bar{\Omega};$$

`flagm=True` returns only M_t ; otherwise the pair (X_t, M_t) is returned.

Multivariate case. Two interchangeable methods:

- **"Cholesky"**: builds the full $(d \cdot \text{size}) \times (d \cdot \text{size})$ covariance matrix from Eq. (4) evaluated at all pairwise time differences, draws one multivariate normal sample, and reshapes into d component paths. Exact but $O((d \cdot \text{size})^3)$ in cost.
- **"fft"** (default): builds the joint 2×2 spectral density matrix

$$S(f) = \begin{pmatrix} S_{11}(f) & S_{12}(f) \\ S_{21}(f) & S_{22}(f) \end{pmatrix}, \quad S_{ij} = \text{FFT}(C_{ij}) \cdot dt$$

(Wiener–Khinchin theorem) on a padded, symmetric lag grid; enforces Hermitian symmetry $S(-f) = \overline{S(f)}$; draws Hermitian-symmetric complex Gaussian noise Z_k ; performs the spectral factorisation $S_k = H_k H_k^*$ via eigendecomposition (regularised by $10^{-14}I$); forms $\hat{X}_k = H_k Z_k$; and inverts via FFT, cropping the central quarter of the resulting path to avoid circular boundary artefacts.

2.2.8 LogSfBM_Nested_Simulation(size, subsample, sigma, flagm, modevol)

A "nested" variant in which ω may be supplied externally via `modevol` (e.g. from an outer-layer Log-SfBM), enabling hierarchical multifractal constructions. The price/variance formulas use the *full* exponential (not the half-exponential of the standard MRW):

$$\tilde{X}_t = \int_0^t e^{\omega_s} dW_s \quad (\text{nested MRW}), \quad (10)$$

$$\tilde{M}_t = \sigma^2 \int_0^t e^{2\omega_s} ds \quad (\text{nested MRM}). \quad (11)$$

Note the factor of 2 in the exponent of $\tilde{M}_t$, reflecting the squared-volatility convention of the nested model.

2.2.9 `genlogVol(size, subsample, M)` and `genlogVol_New_perscale(size, subsample, M, scale, sigma, flagm)`

Generate a log-realised-volatility *proxy* via quadratic-variation (QV) aggregation. Given a fine-grid price path X_t with step $1/M$ (or $1/(M \cdot \text{scale})$ in the rescaled variant), the aggregated QV at coarse step k is

$$\text{QV}_k = \sum_{j=(k-1)M}^{kM-1} (X_{(j+1)/M} - X_{j/M})^2, \quad \ell_k = \log(\text{QV}_k) - \overline{\log(\text{QV})}. \quad (12)$$

The analogous log integrated-variance proxy is built from the MRM increments in the same way. Numerical $\pm\infty$ values (from $\text{QV} = 0$) are replaced by linear interpolation before mean-centring.

3 Module: LogSfBM_StatisticalProperties.py — Statistical Diagnostics

This module implements thirteen functions organised into four families: increment generation, distributional diagnostics, second-order (autocovariance) diagnostics, and multifractal moment-scaling diagnostics. All functions operate on increments

$$\delta_\tau X_t = X_{t+\tau} - X_t$$

of one of four underlying processes: the MRW price path, the MRM integrated variance, the raw field ω_t (“S-fbm”), or the log-realised-variance QV proxy (“log MRM”).

3.1 Increment generation

3.1.1 Increments(model, tau, size, subsample, logincr_flag, type_inc, marketlogvol)

The shared primitive used throughout the module. If `marketlogvol` is supplied (non-empty), increments are computed directly from this empirical series, bypassing simulation entirely. Otherwise, `type_inc` selects the simulated process:

type_inc	Underlying process
"increments MRW"	$X_t = \text{MRW price path}$
"increments MRM"	$X_t = \text{MRM integrated variance}$
"S-fbm"	$X_t = \omega_t$ (raw log-vol field)
"log MRM"	$X_t = \text{log-realised-variance QV proxy}$

In all cases $\delta_\tau X_t = X_{t+\tau} - X_t$ is returned, or $\log |\delta_\tau X_t|$ if `logincr_flag=True`.

3.2 Distributional diagnostics

3.2.1 LogIncrementsDistributionDensityAcrossScale(model, taulist, size, subsample, logincr_flag, savepath)

For each $\tau \in \text{taulist}$, standardises $\tilde{\delta}_\tau X_t = \delta_\tau X_t / \sqrt{\text{Var}(\delta_\tau X_t)}$ and estimates its density via a Gaussian kernel density estimator,

$$\hat{p}_\tau(x) \approx \frac{1}{Nh} \sum_i K\left(\frac{x - x_i}{h}\right),$$

plotting $\log \hat{p}_\tau(x)$ (vertically offset per scale for legibility) against the standard normal reference $\log [(2\pi)^{-1/2} e^{-x^2/2}]$. Returns `None`; the function is purely visual.

3.3 Multifractal moment-scaling diagnostics

3.3.1 MomentIncrementsRepresentation(logsfbm, q_list, tau_list, size, subsample, type_inc)

Estimates, for each moment order q and scale τ , the q -th absolute moment

$$M_q(\tau) = \mathbb{E}[|\delta_\tau X_t|^q] \approx \frac{1}{N_\tau} \sum_{k=0}^{N_\tau-1} |\delta_\tau X_{k\tau}|^q, \quad N_\tau = \left\lfloor \frac{N}{\tau} \right\rfloor, \quad (13)$$

and fits the log-log power-law scaling relation $\log M_q(\tau) = \zeta(q) \log \tau + c_q$ by ordinary least squares, where the slope $\zeta(q)$ is the empirical multifractal structure-function exponent.

3.3.2 MomentIncrementsRepresentationSimulatedVSTheoretical(logsfbm, q_list, tau_list, size, subsample, type_inc)

Computes the same empirical moments $M_q(\tau)$ as above, then compares them against a closed-form theoretical prediction that depends on `type_inc`:

MRW ("increments MRW"):

$$\log M_q(\tau) = \frac{q}{2} \log \tau + \frac{\lambda^2}{\tau^2} \frac{q}{2} \left(\frac{q}{2} - 1 \right) \left[\frac{\tau^2}{2H(1-2H)} - \frac{2\tau^{2H+2}}{4HT^{2H}(1-4H^2)(1+H)} \right] + \frac{q}{2} \log 2 + \log \frac{\Gamma(\frac{q+1}{2})}{\sqrt{\pi}}. \quad (14)$$

MRM ("increments MRM"):

$$\log M_q(\tau) = q \log \tau + \frac{\lambda^2}{2\tau^2} q(q-1) \left[\frac{\tau^2}{2H(1-2H)} - \frac{2\tau^{2H+2}}{4HT^{2H}(1-4H^2)(1+H)} \right]. \quad (15)$$

S-fbm ("S-fbm"):

$$\log M_q(\tau) = qH \log \left(\frac{\tau}{T} \right) + \frac{q}{2} \log \left(\frac{2\lambda^2}{H(1-2H)} \right) + \log \left(\Gamma \left(\frac{q+1}{2} \right) \right) - \frac{1}{2} \log(\pi). \quad (16)$$

log MRM ("log MRM"): an ordinary-least-squares log-log regression of the *empirical* curve itself is plotted as a reference line.

3.3.3 MomentIncrementsEvolutionWrtParameter(logsfbm, q, tau_list, size, subsample, type_inc, parameter)

Studies the sensitivity of the moment-scaling curve $M_q(\tau)$ (and its exponent $\zeta(q)$) to the model parameters H and λ^2 , by sweeping one while holding the other fixed. The `parameter` argument is a dictionary with key "H" or "lambdasquare" mapping to a list of values to sweep; the missing key defaults to a single-element list holding the model's current value for that parameter.